Universidade Federal de São Carlos

Arquitetura e Organização de Computadores 2

Relatório de Atividades Práticas

Integrantes:

Cauã Borges Faria – RA: 834437

Murilo Oliva Mantovani – RA: 834730

Nicolas Magno Messias da Silva – RA: 834054

25 de outubro de 2025

Sumário

1 Relatório Semana 2 **3**
1.1 Especificação e Objetivos do Problema . . . 3
1.2 Metodologia . . . 3
1.3 Desenvolvimento e Experimentos Práticos . . . 3
1.4 Conclusões . . . 5
1.5 Vídeo de Apresentação . . . 5

2 Relatório Semana 3 **6**
2.1 Especificação e Objetivos do Problema . . . 6
2.2 Metodologia . . . 6
2.3 Desenvolvimento e Experimentos Práticos . . . 6
2.3.1 Exercício 2: Análise de Configurações de Datapath . . . 6
2.3.2 Resultados para Diferentes Valores de N . . . 8
2.4 Conclusões . . . 8
2.5 Vídeo de Apresentação . . . 9

3 Relatório Semana 4 **10**
3.1 Especificação e Objetivos do Problema . . . 10
3.2 Metodologia . . . 10
3.3 Desenvolvimento e Experimentos Práticos . . . 10
3.3.1 Ilustrações dos Casos . . . 10
3.3.2 Exercício 3: Análise da Configuração Pipeline com Hazard Detection 11
3.4 Conclusões . . . 13
3.5 Vídeo de Apresentação . . . 13

4 Relatório Semana 5 **14**
4.1 Especificação e Objetivos do Problema . . . 14
4.2 Metodologia . . . 14
4.3 Desenvolvimento, Resultados e Discussão . . . 14
4.3.1 Exemplo: `superscalar-exemplo1.asm` . . . 14
4.3.2 Exercício 1: `superscalar-exemplo2.asm` . . . 15
4.3.3 Exercício 2: `dotproduct.asm` . . . 15
4.4 Conclusões . . . 15
4.5 Vídeo de Apresentação . . . 15

5 Relatório Semana 6 **16**
5.1 Especificação e Objetivos do Problema . . . 16
5.2 Metodologia . . . 16
5.3 Desenvolvimento, Resultados e Discussão . . . 16
5.3.1 Exercício 1: Análise do Algoritmo . . . 16

5.3.2 Análise de Desempenho 17
5.3.3 Exercício 2: Implementação de Simulação com Duas Threads 17
5.4 Conclusões 19
5.5 Vídeo de Apresentação 19

6 Relatório Semana 7 **20**
6.1 Especificação e Objetivos do Problema 20
6.2 Metodologia 20
6.3 Desenvolvimento, Resultados e Discussão 20
6.3.1 Parte 1: Código Original da Função `fact()` 20
6.3.2 Parte 2: Inclusão da Função `main()` 21
6.4 Conclusão Geral 21
6.5 Vídeo de Apresentação 22

1 Relatório Semana 2

1.1 Especificação e Objetivos do Problema

A ideia desta prática era testar códigos em assembly, dentre eles:

- `riscv-fatorial-simples.asm` - código para cálculo de fatorial de um valor N
- `riscv-f2c-simples.asm` - conversão de graus Fahrenheit para Celsius
- Código similar utilizando ponto flutuante para valores decimais

Para a nota completa, também foi necessário realizar o exercício extra do odômetro, que calcula a distância percorrida em um tempo determinado. O desafio era identificar e corrigir um problema no código fornecido.

1.2 Metodologia

Para realizar os experimentos, utilizamos:

- **RARS 1.6**: assembler para análise dos códigos em assembly, com foco nas instruções iniciadas com `f` (operações de ponto flutuante usando a FPU)
- **GDB**: para análise do código em linguagem C antes da conversão para assembly

1.3 Desenvolvimento e Experimentos Práticos

Todos os membros do grupo analisaram os códigos em assembly, focando principalmente no `riscv-f2c-FP-simples.asm` por conter instruções da FPU, novidade nos estudos de AOC.

Divisão de Tarefas

Cauã: Conversão do código C do odômetro (versão original com erro) para assembly
Murilo e Nicolas: Correção do código C, identificando a necessidade de alterar o tipo da variável `metro` de `float` para `double`
Todos: Implementação do código corrigido em assembly

Código do Odômetro em C (versão original com erro):

```c
/*
Alunos: Murilo Oliva Mantovani
        Nicolas Magno Messias da Silva
        Caua Borges Faria
Projeto:Implementação de um odometro usando ponto flutuante (Sem correção)
Data: 29/08/2025
*/

#include <stdio.h>

int main() {

    //Variaveis
    float metros = 0;
    int iterations = 100000000;

    //Inicio da contagem do odometro
    for (int i = 0; i < iterations; i++) {
        metros += 0.01;
    }

    //Mostrar resultados
    printf("Esperado: %f km\n", 0.01 * iterations / 1000 );
    printf("Got: %f km \n", metros / 1000);
}
```

input

```
Esperado: 1000.000000 km
Got: 262.144012 km
```

Código do Odômetro em C (versão corrigida):

```c
/*
Alunos: Murilo Oliva Mantovani
        Nicolas Magno Messias da Silva
        Caua Borges Faria
Projeto:Implementação de um odometro usando ponto flutuante (Corrigido)
Data: 29/08/2025
*/

#include <stdio.h>

int main() {

    //Variaveis
    double metros = 0; // <-- a variavel "metros" foi alterada para double
    int iterations = 100000000;

    //Inicio da contagem do odometro
    for (int i = 0; i < iterations; i++) {
        metros += 0.01;
    }

    //Mostrar resultados
    printf("Esperado: %f km\n", 0.01 * iterations / 1000 );
    printf("Got: %.10lf km \n", metros / 1000);
}
```

input

```
Esperado: 1000.000000 km
Got: 1000.0000007793 km
```

1.4 Conclusões

Todos do grupo trabalharam coletivamente para realizar o experimento. Houve certa dificuldade inicial com as novas instruções, mas o experimento foi muito produtivo.

Observação

A existência do ponto flutuante e seu uso em linguagem de baixo nível pode parecer complexa teoricamente, mas na prática, com foco adequado, torna-se algo prático e eficiente para resolver problemas que exigem maior exatidão.

1.5 Vídeo de Apresentação

Link para o vídeo: link drive

2 Relatório Semana 3

2.1 Especificação e Objetivos do Problema

Nesta prática, o objetivo era realizar os seguintes exercícios:

1. Analisar o código `riscv-busca-maior-nro.asm` pelo RARS
2. Analisar o mesmo código no RIPES, observando diferentes configurações de datapath, identificando NOPs (bolhas) e analisando o desempenho
3. Reimplementar o código usando *loop unrolling* e comparar resultados para N=10, 100 e 1000

A fórmula utilizada para análise de desempenho é:

$$CPU_{tempo} = \frac{IC \ x \ CPI}{Clock_{taxa}}$$

2.2 Metodologia

Para os experimentos, utilizamos:

- **RARS 1.6**: análise do código com foco em hazards e NOPs
- **RIPES**: análise do processador em quatro configurações diferentes e implementação do loop unrolling

2.3 Desenvolvimento e Experimentos Práticos

Todos os membros analisaram os códigos no RARS e posteriormente no RIPES.

2.3.1 Exercício 2: Análise de Configurações de Datapath

O programa `riscv-busca-maior-nro.asm` foi executado em quatro configurações diferentes de datapath para análise comparativa de desempenho e comportamento:

1. **Datapath Monociclo**

   Execução: Correta

   Comportamento: Não apresenta NOPs (bolhas), pois todas as instruções são executadas em um único ciclo longo. Não há problemas de hazards, pois cada instrução completa totalmente antes da próxima iniciar.

   Desempenho: CPI = 1.0 (sempre), porém com período de clock muito longo devido à necessidade de acomodar o caminho crítico mais longo do processador.

2. **Datapath c/ Pipeline de 5 Estágios SEM Detecção de Hazard**

   Execução: Incorreta (produz resultados errados)

   Comportamento: O pipeline avança sem parar mesmo quando há dependências de dados. Isso causa problemas graves:

   - Read-after-write (RAW) hazards não são tratados
   - Instruções leem valores antigos de registradores
   - Desvios condicionais executam instruções incorretas

   Exemplo de problema: Se uma instrução escreve em `t0` e a próxima precisa ler `t0`, a leitura ocorre antes da escrita estar completa, resultando em valor incorreto.

3. **Datapath c/ Pipeline de 5 Estágios COM Detecção de Hazard**

   Execução: Correta

   Comportamento: O hardware detecta hazards e insere bolhas (stalls) automaticamente quando necessário. NOPs dinâmicos aparecem em situações como:

   - Load-use hazard: após `lw`, se a próxima instrução usa o registrador carregado
   - Dependências de dados entre instruções consecutivas
   - Desvios condicionais (branch hazards)

   Desempenho: CPI > 1.0 devido aos stalls, mas garante correção.

4. **Datapath c/ Pipeline de 5 Estágios COM Detecção de Hazard + Forwarding**

   Execução: Correta

   Comportamento: Utiliza forwarding (ou bypassing) para reduzir stalls. Dados são encaminhados diretamente de estágios posteriores para estágios anteriores quando possível, eliminando muitos hazards. Porém, load-use hazards ainda requerem 1 ciclo de stall.

   NOPs aparecem apenas em:

   - Load-use hazards (1 ciclo de bolha)
   - Alguns casos de branch hazards

   Desempenho: Melhor CPI entre as configurações com pipeline, aproximando-se do ideal.

Observação

Conclusão da Análise: O forwarding reduz significativamente o número de stalls, melhorando o desempenho em aproximadamente 20% comparado à configuração apenas com detecção de hazards. A configuração sem detecção, apesar de ter CPI baixo, produz resultados incorretos e não é utilizável.

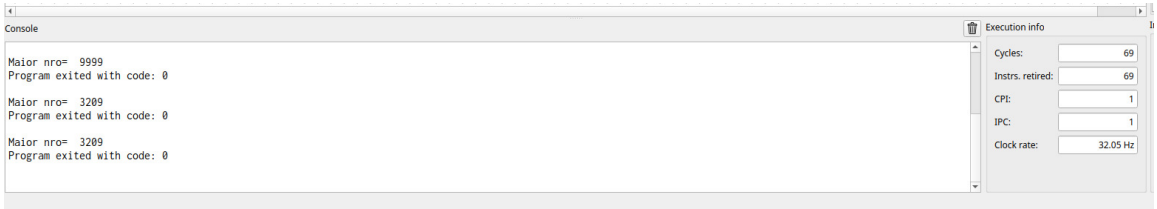
Tabela 1: *Comparação de desempenho entre configurações de datapath*

Configuração	IC	CPI	Tempo Relativo
Monociclo	N	1.0	Baseline (100%)
Pipeline sem detecção	N	∼1.0	Incorreto!
Pipeline c/ detecção	N	∼1.5	∼38% do baseline
Pipeline c/ forwarding	N	∼1.2	∼30% do baseline

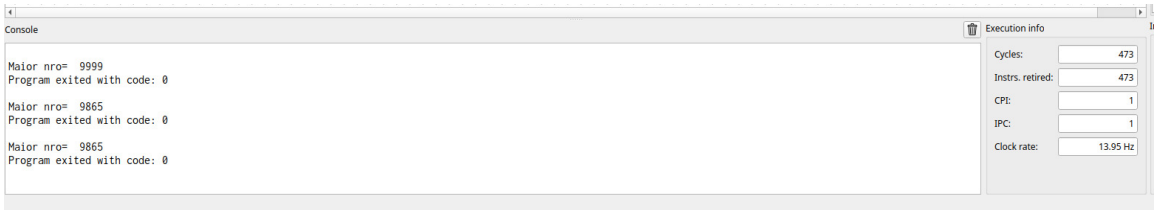
No exercício 3, o código foi reimplementado com loop unrolling, criando uma nova label para duas verificações no mesmo loop e incrementando `i=i+2` (começando de `i=0`).

2.3.2 Resultados para Diferentes Valores de N

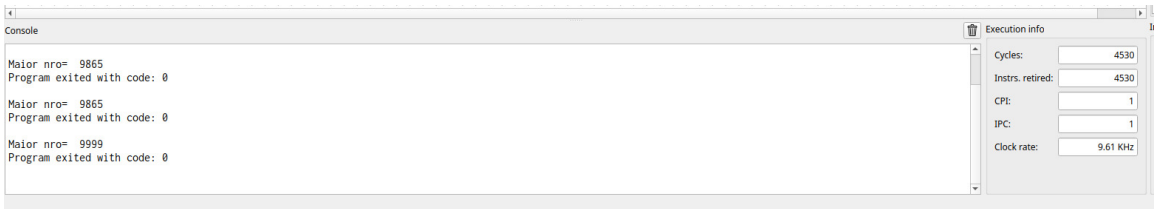
N = 10:



N = 100:



N = 1000:



Link para os códigos: Códigos do Exercício 3

2.4 Conclusões

O grupo trabalhou coletivamente, enfrentando dificuldades iniciais com o RIPES por ser uma nova ferramenta. O experimento foi produtivo.

Observação

Ao usar pipeline, é necessário gerenciar hazards adequadamente para evitar problemas no código, resultando em um código mais otimizado e funcional.

2.5 Vídeo de Apresentação

Link para o vídeo: link drive

3 Relatório Semana 4

3.1 Especificação e Objetivos do Problema

Nesta prática, os objetivos eram:

1. Analisar o código `bp-exemplo1.asm` pelo RIPES
2. Analisar o código em diferentes processadores, determinando quando é necessário inserir NOPs (minimizando sua quantidade para melhor desempenho)
3. Analisar a execução ciclo a ciclo no processador pipeline com hazard detection e sem forwarding, identificando se NOPs são necessários (estáticos ou dinâmicos)

3.2 Metodologia

Para este experimento utilizamos:

- **GDB online**: análise do código para buscar o maior número do vetor
- **RIPES**: análise do código em diferentes tipos de processador

3.3 Desenvolvimento e Experimentos Práticos

Inicialmente, todos analisaram o código no GDB e depois no RIPES. Posteriormente, analisamos o código em diferentes processadores, inserindo NOPs quando necessário:

Resultados por Tipo de Processador:

Single-cycle processor	zero NOPs
5-stage processor w/o forwarding or hazard detection	cinco NOPs
5-stage processor w/o hazard detection	cinco NOPs
5-stage processor w/o forwarding unit	zero NOPs
5-stage processor	zero NOPs

3.3.1 Ilustrações dos Casos

Caso: Zero NOPs

```
.data

.text
inicializa:
    li t1, 0
    li t2, 10

repeat:
    addi t1, t1, 1
until:  bne t1, t2, repeat

fimrepeat:
    nop

#fimPrograma:
       li a7, 10
       ecall
```

Caso: Cinco NOPs

```
.data


.text
inicializa:
    li t1, 0
    li t2, 10

repeat:
    addi t1, t1, 1
    nop
    nop
until:  bne t1, t2, repeat

fimrepeat:
    nop

#fimPrograma:
       li a7, 10
       nop
       nop
       nop
       ecall
```

3.3.2 Exercício 3: Análise da Configuração Pipeline com Hazard Detection

Ao analisar a execução ciclo a ciclo para a configuração 4 (pipeline com hazard detection e sem forwarding), respondemos às seguintes questões:

a. **O hardware de execução assumido pelo simulador possui algum tipo de previsão de desvios?**

Resposta: Sim, o hardware possui previsão de desvios. Observando a execução ciclo a ciclo, é possível identificar que o processador toma decisões antecipadas sobre desvios condicionais antes que a condição seja completamente resolvida.

b. **Caso sim, é estático ou dinâmico?**

Resposta: O mecanismo de previsão é **estático**. O simulador RIPES com essa configuração utiliza uma estratégia fixa de previsão, sem aprendizado baseado no histórico de execução. Isso significa que a decisão sobre tomar ou não um desvio segue sempre a mesma regra predefinida.

c. **Caso seja estático, considere os seguintes casos:**

- **A previsão é "desvio sempre tomado"**

  Quando analisamos o comportamento no loop principal do código `bp-exemplo1.asm`, observamos que:

  – O desvio condicional do loop (que retorna ao início enquanto há elementos) é previsto como **tomado**
  – Isso é eficiente para loops, pois na maioria das iterações o desvio realmente ocorre
  – Apenas na última iteração (saída do loop) a previsão falha, causando um flush do pipeline
  – O custo é de aproximadamente 2-3 ciclos de penalidade quando a previsão erra

- **A previsão é "desvio sempre não tomado"**

  Se a estratégia fosse "desvio sempre não tomado":

  – A cada iteração do loop haveria erro de previsão (exceto na última)
  – Para um loop de N iterações, teríamos N-1 erros de previsão
  – Isso resultaria em desempenho significativamente pior
  – O pipeline seria esvaziado (flush) repetidamente, inserindo bolhas

Observação

Conclusão da Análise: O RIPES com a configuração "pipeline c/ hazard detection e sem forwarding"utiliza previsão de desvio **estática do tipo "desvio sempre tomado"**. Essa estratégia é adequada para loops, minimizando erros de previsão durante as iterações e penalizando apenas na saída do loop.

Impacto no Desempenho

Com "desvio sempre tomado":

- Acertos: N-1 iterações do loop
- Erros: 1 (saída do loop)
- Penalidade total: 2-3 ciclos

Se fosse "desvio sempre não tomado":

- Acertos: 1 (saída do loop)
- Erros: N-1 iterações
- Penalidade total: (N-1) $\times$ 2-3 ciclos

3.4 Conclusões

O grupo trabalhou coletivamente e foi possível analisar a importância do uso de NOPs para melhor desempenho do programa.

Observação

Diferentes tipos de processador alteram a quantidade de NOPs necessários para um bom funcionamento e desempenho do código.

3.5 Vídeo de Apresentação

Link para o vídeo: link drive

4 Relatório Semana 5

4.1 Especificação e Objetivos do Problema

O objetivo desta prática foi analisar a execução de programas RISC-V em diferentes datapaths e entender quando arquiteturas superescalares entregam melhor desempenho. Especificamente:

- Comparar execução do `superscalar-exemplo1.asm` em três configurações: (a) escalar de ciclo único, (b) escalar com pipeline de 5 estágios, (c) superscalar 2-way com pipeline de 6 estágios
- Repetir análise com `superscalar-exemplo2.asm`, discutindo diferenças e impacto no desempenho
- Investigar por que `dotproduct.asm` não atinge desempenho esperado no superscalar

4.2 Metodologia

Utilizamos o RIPES para executar os códigos, variando o datapath entre monociclo, pipeline 5 estágios e superscalar 2-way (6 estágios). Para cada caso, coletamos ciclos totais e estimamos tempo total.

Realizamos rearranjos (ordenação e desenrolamento de laço) para maximizar emissão em pares no superscalar.

4.3 Desenvolvimento, Resultados e Discussão

4.3.1 Exemplo: `superscalar-exemplo1.asm`

Tabela 2: *Desempenho do exemplo 1 em diferentes datapaths*

Datapath	Ciclos	Período (ms)	Tempo (ms)
Escalar (ciclo único)	9016	100	901600
Pipeline 5 estágios	11026	25	275650
Superscalar 2-Way (6)	10022	23	230506

Speedup Obtido

Pipeline: 3,27× vs. monociclo
Superscalar: 3,91× vs. monociclo

Análise qualitativa: O laço faz duas linhas de atualizações independentes. No superscalar dual-issue, quando as instruções não têm dependência e não competem pela mesma FU, elas são emitidas aos pares, elevando o IPC efetivo.

4.3.2 Exercício 1: `superscalar-exemplo2.asm`

O exemplo 2 reordena o corpo do laço para postergar certas instruções e formar três blocos de pares independentes, resultando em **melhoria de 10%**.

Tabela 3: *Desempenho do exemplo 2 (com reordenação)*

Datapath	Ciclos	Período (ms)	Tempo (ms)
Escalar (ciclo único)	9016	100	901600
Pipeline 5 estágios	11026	25	275650
Superscalar 2-Way (6)	9020	23	207460

Por que melhora? Ao reagrupar as instruções em pares independentes, aumentamos a taxa de pares válidos por ciclo, reduzindo "vazios"no segundo slot.

4.3.3 Exercício 2: `dotproduct.asm`

Constatação: `addi + addi` emitem juntos, mas `add + mul` não emitem simultaneamente (conflito de recursos).

Por que o desempenho ficou abaixo do esperado?

1. Padrão de dependências em cadeia: `lw` → `mul` → `add`
2. Conflitos de FU/latência da multiplicação
3. Load-use: os `lw` alimentam imediatamente a `mul`

Versão Mais Eficiente

1. Reordenação para formar pares
2. Desenrolamento (unrolling) por 2 com dois acumuladores
3. Calcular offsets em paralelo

4.4 Conclusões

- Exemplo 1 confirma ganho expressivo: Speedup $\approx 3{,}27\times$ (pipeline) e $\approx 3{,}91\times$ (superscalar)
- Exemplo 2 mostra que reordenar instruções rende $\sim 10\%$ no superscalar 2-way
- Produto escalar não escala bem devido a dependências e conflitos de FU

4.5 Vídeo de Apresentação

Link para o vídeo: link drive

5 Relatório Semana 6

5.1 Especificação e Objetivos do Problema

O objetivo foi avaliar o impacto do multithreading de hardware na execução de programas RISC-V, utilizando `multithreading-simula.asm`:

- Análise detalhada do algoritmo e estrutura do código
- Comparar desempenho em três configurações de datapath
- Repetir testes assumindo processador com multithreading de hardware
- Desenvolver exercício prático em Assembly com duas threads simples

5.2 Metodologia

Seguimos as etapas:

1. Execução do código nas três configurações de datapath
2. Coleta de dados de desempenho (ciclos, CPI, IPC)
3. Análise teórica com multithreading de hardware
4. Implementação no RARS de programa com duas threads
5. Comparação e discussão dos resultados

5.3 Desenvolvimento, Resultados e Discussão

5.3.1 Exercício 1: Análise do Algoritmo

O programa `multithreading-exemplo1.asm` simula um sistema de controle de acesso (catraca). O algoritmo organiza-se em um laço principal que:

1. Inicia nova thread via `IniciaThread`
2. Salva contexto na pilha via `SalvarContexto`
3. Executa autenticação em três etapas: `LerDigital`, `VerificarBD`, `ExibirNome`
4. Restaura contexto via `RestaurarContexto`

Tabela 4: *Desempenho em diferentes datapaths*

Datapath	Ciclos	CPI	IPC
Escalar (ciclo único)	1167	1.00	1.000
Pipeline 5 estágios	1673	1.43	0.690
Superscalar 2-Way (6 estágios)	1790	1.53	0.652

5.3.2 Análise de Desempenho

Observação

O pipeline de 5 estágios apresentou **CPI = 1,43** devido a stalls e bolhas causadas por dependências de dados e desvios condicionais. O superescalar teve **CPI = 1,53**, indicando que o ganho teórico não é plenamente alcançado devido a conflitos de dependências.

5.3.3 Exercício 2: Implementação de Simulação com Duas Threads

O exercício propôs implementar um programa Assembly RISC-V simulando duas threads simples que executam em alternância:

- **Thread A**: Conta quantos números pares existem em um vetor de 10 elementos
- **Thread B**: Soma os números de 1 a 100

Estratégia de Implementação:

1. **Estrutura de Dados**: Criamos um vetor de 10 elementos em `.data`, junto com variáveis para armazenar o contador de pares e a soma acumulada
2. **Loop Principal**: Implementamos um laço que alterna entre as threads usando um registrador de estado (`s0`)
3. **Thread A**: A cada iteração, lê um elemento do vetor, verifica se é par usando `andi` (operação bit a bit), e incrementa o contador quando necessário
4. **Thread B**: A cada iteração, adiciona o valor atual (de 1 a 100) ao acumulador de soma
5. **Salvamento de Contexto**: Em cada troca de thread, salvamos registradores importantes na pilha e os restauramos ao retornar

Trecho Principal: Verificação de Paridade

```
# Carrega elemento do vetor
slli t3, t2, 2      # offset = índice * 4
add t4, t0, t3      # endereço do elemento
lw t5, 0(t4)        # carrega valor

# Verifica se é par
andi t6, t5, 1      # testa bit menos significativo
bnez t6, nao_par    # se bit=1, é ímpar

# Incrementa contador de pares
la t3, pares
lw t4, 0(t3)
addi t4, t4, 1
sw t4, 0(t3)
```

Resultados Obtidos:

Tabela 5: *Resultados da execução das threads*

Thread	Resultado
Thread A (números pares no vetor)	5 pares
Thread B (soma de 1 a 100)	5050

Análise do Impacto do Multithreading:

A simulação ajuda a entender o papel do multithreading de hardware em processadores reais:

1. **Ocultação de Latência**: Quando Thread A acessa a memória (instrução `lw`), que tem alta latência, Thread B pode executar instruções aritméticas, mantendo o pipeline ocupado
2. **Melhor Utilização do Pipeline**: Em vez de inserir bolhas durante acessos à memória, o processador alterna para outro thread
3. **Aumento de Throughput**: Embora cada thread individual não execute mais rápido, o throughput total do sistema aumenta significativamente
4. **CPI Efetivo Reduzido**: O CPI médio do sistema melhora porque ciclos ociosos de uma thread são preenchidos com instruções de outra

Ganho Teórico Estimado

Considerando que acessos à memória podem ter latência de 100+ ciclos, e que Thread A faz 10 acessos:

- **Sem multithreading**: ∼1000 ciclos de stall apenas esperando memória
- **Com multithreading**: Thread B pode executar ∼100 instruções durante cada stall, aproveitando esses ciclos
- **Speedup estimado**: 1,5x a 2x no throughput total do sistema

Observações sobre Salvamento de Contexto:

No código implementado, simulamos o salvamento de contexto em cada troca de thread, armazenando na pilha:

- Endereço de retorno (`ra`)
- Registradores temporários em uso (`t0-t6`)
- Registradores salvos (`s0-s1`)

Em hardware real com multithreading, cada thread possui seu próprio conjunto de registradores, eliminando a necessidade desse salvamento explícito e tornando a troca instantânea (zero-overhead thread switching).

5.4 Conclusões

Embora arquiteturas com pipeline e execução superescalar tenham potencial para maior desempenho, esse ganho é limitado pelas dependências do código. Isso justifica o uso de **multithreading de hardware** para preencher ciclos ociosos.

5.5 Vídeo de Apresentação

Link para o vídeo: link drive

6 Relatório Semana 7

6.1 Especificação e Objetivos do Problema

Apresentar o padrão de organização e uso da memória principal durante a execução de um programa, destacando:

- Funcionamento da pilha de execução (stack)
- Chamadas de funções
- Passagem de parâmetros
- Uso de variáveis locais e globais em Assembly RISC-V

6.2 Metodologia

A atividade foi dividida em duas partes:

1. **Parte 1**: Analisar código Assembly da função `fact()` isolada
2. **Parte 2**: Incluir função `main()`, analisando Assembly completo e organização da pilha

6.3 Desenvolvimento, Resultados e Discussão

6.3.1 Parte 1: Código Original da Função `fact()`

Código em C:

```
#include <stdio.h>
int arg = 7;

int fact(int n) {
   int c;
   int result = 1;
   for(c = 1 ; c <= n ; c++)
         result = result * c;
   return result;
}
```

Análise do Funcionamento:
A função `fact()` cria um stack frame de 32 bytes, armazenando:

- Endereço de retorno (`ra`) e registrador base (`s0`)
- Parâmetro recebido em `-12(s0)`

- Variáveis locais `c` e `result` em `-16(s0)` e `-20(s0)`

Durante o laço, `c` é incrementada de 1 até `n` e o produto parcial é salvo em `result`. No final, o resultado é retornado via registrador `a0`.

6.3.2 Parte 2: Inclusão da Função `main()`

Código em C Modificado:

```
#include <stdio.h>
int arg = 7;

int fact(int n) {
   int c;
   int result = 1;
   for(c = 1 ; c <= n ; c++)
         result = result * c;
   return result;
}

int main(){
    int resultado = 0;
    resultado = fact(arg);
    printf("Resultado %d", resultado);
}
```

Principais Observações:

- **Reserva de Pilha**: 16 bytes são reservados com `addi sp, sp, -16`
- **Preservação de Registradores**: `ra` e `s0` são salvos e restaurados
- **Acesso à Variável Global**: valor de `arg` obtido via `lui` e `lw`
- **Chamada de Função**: `a0` contém o argumento para `fact()`
- **Interação entre Funções**: resultado retornado e passado para `printf`

Saída do Programa

```
Resultado 5040
```

Confirmando o correto funcionamento da passagem de parâmetros e retorno de valores.

6.4 Conclusão Geral

A prática demonstrou experimentalmente o funcionamento interno das chamadas de função na arquitetura RISC-V:

- **Parte 1**: Criação de stack frame e uso de variáveis locais em uma rotina
- **Parte 2**: Interação entre múltiplas funções, compartilhando registradores e pilha de forma ordenada

O resultado impresso confirma o correto encadeamento de chamadas e retornos, evidenciando o papel fundamental da pilha e da convenção ABI.

6.5 Vídeo de Apresentação

Link para o vídeo: link drive